

画栅栏

题目大意：给定起点，然后 n 次从上一次的终点开始往左或者向右涂色，问被涂次数大于等于 k 次的长度一共为多少。 $n \leq 1e5$, 距离 $\leq 1e9$

输入：

6 2

2 R

6 L

1 R

8 L

1 R

2 R

输出：

6

解题思路

这道题看上去是一道比较水的题，但是仔细观察数据范围之后，我们会发现没有办法直接开标记数组，会炸空间。

我们很容易想到一种办法：区间离散化。因为只有 $1e5$ 次操作，所以最多只有 $2e5$ 个点被遍历到，其他点都不会被访问到，如果全部都标记会浪费很大的空间，所以需要路径压缩，也就是离散化

区间离散化

有一条1—10的数轴，长度为9，给定四个区间 $[2, 4]$, $[3, 6]$, $[8, 10]$, $[6, 9]$. 我们来尝试把区间压缩一下。也就是离散化一下。

(1) 开一个数组记录一下所有的端点

(2) 对这些端点排序 2 3 4 6 6 8 9 10

(3) 为了提高效率和正确性，我们还要去重

2 3 4 6 8 9 10

(4) 离散化转换成 1 2 3 4 5 6 7

(5) 原区间转换成 $[1, 3]$, $[2, 4]$, $[5, 7]$, $[4, 6]$

我们会发现覆盖关系并没有改变。



区间离散化

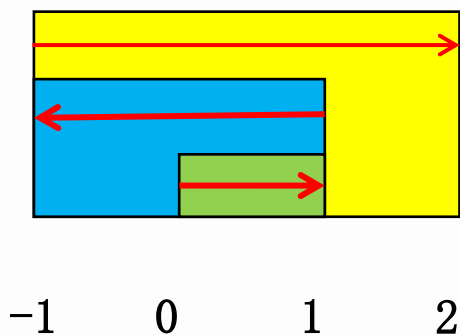
看上去离散化比较简单，但是实际操作过程中有一点麻烦，并且由于原区间和离散化的区间的长度在表现形式上是不相等的，所以还需要计算区间长度。我们这里介绍一种新的方法——扫描线。

但是区间离散化也是很重要的一个操作。必须要掌握。



扫描线

每一次操作都会有一个区间，我们把每个区间看做一个矩形，那么涂色的次数就是矩形重叠的区域。



输入:

3 2

1 R

2 L

3 R

1、存储矩形竖着的每一条边，由于分左边界(开始)和右边界(结束)，所以我们分别用1和-1来表示，进入矩形，次数+1，离开矩形，次数-1，这样当我遍历完这个矩形的时候，又恢复成0了。

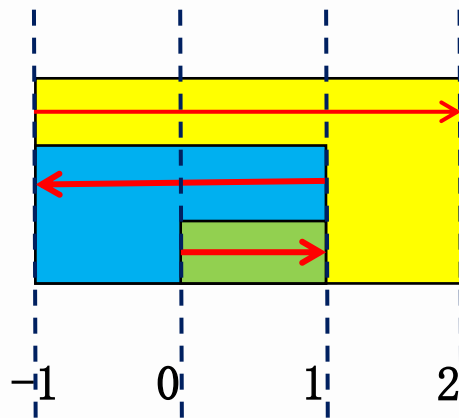
扫描线

2、把线段从左往右排序(根据题目需要)。

(位置, 起点还是终点)

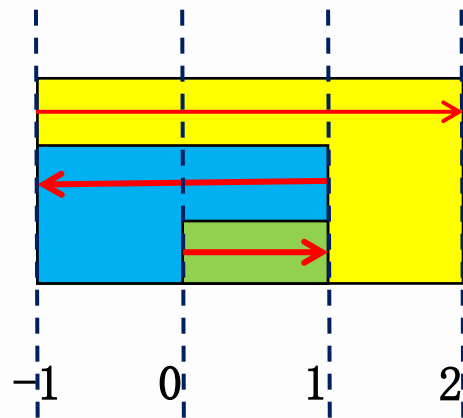
$(-1, 1)$, $(-1, 1)$, $(0, 1)$, $(1, -1)$, $(1, -1)$, $(2, -1)$

3、构造一条竖着的扫描线, 从左往右移动, 移动到第 i 条边的时候, 加上当前这条边的第二个参数。如果 $\geq k$, 那么 i 和 $i-1$ 条边的距离只差就是满足条件的区间。



扫描线

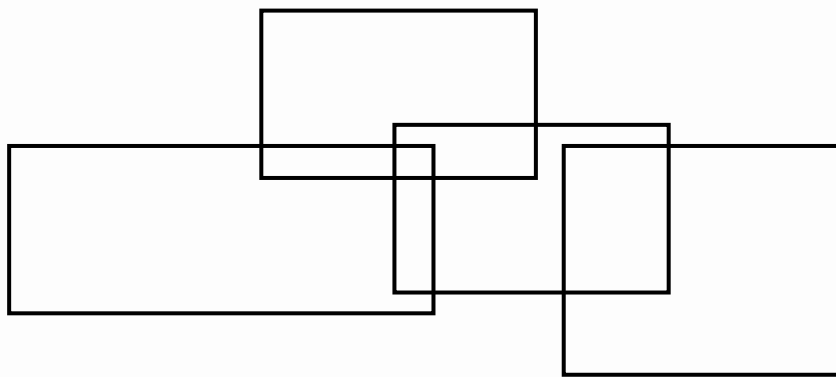
- (1) 初始时 $\text{cnt}=0$ 遍历第一条边 $(-1, 1)$ $\text{cnt}+1$
(2) 此时 $\text{cnt}=1$ 遍历第二条边 $(-1, 1)$ $\text{cnt}+1$
(3) 此时 $\text{cnt}=2 \geq k$ 遍历第三条边 $(0, 1)$, 得到满足条件的区间 $\text{ans}=0-(-1)=1$ $\text{cnt}+1$
(4) 此时 $\text{cnt}=3 \geq k$ $\text{ans}=2$ $\text{cnt}-1$
(5) 此时 $\text{cnt}=2 \geq k$ $\text{ans}=2+0$ $\text{cnt}-1$
(6) 此时 $\text{cnt}=1$ $\text{cnt}-1$
所以最终 $\text{ans}=2$;



```
now=line[1].val;  
for(i=2;i<=1;++i)  
{  
    if(now>=m) ans+=line[i].x-line[i-1].x;  
    now+=line[i].val;  
}
```

矩形面积之和

题目大意：给你 n 个矩形，求这些矩形面积之和
(矩形之间可能相交)。 $N \leq 100000$; 坐标 $\leq 1e9$



矩形面积之和

这是一道离散化+扫描线+线段树的经典题目。

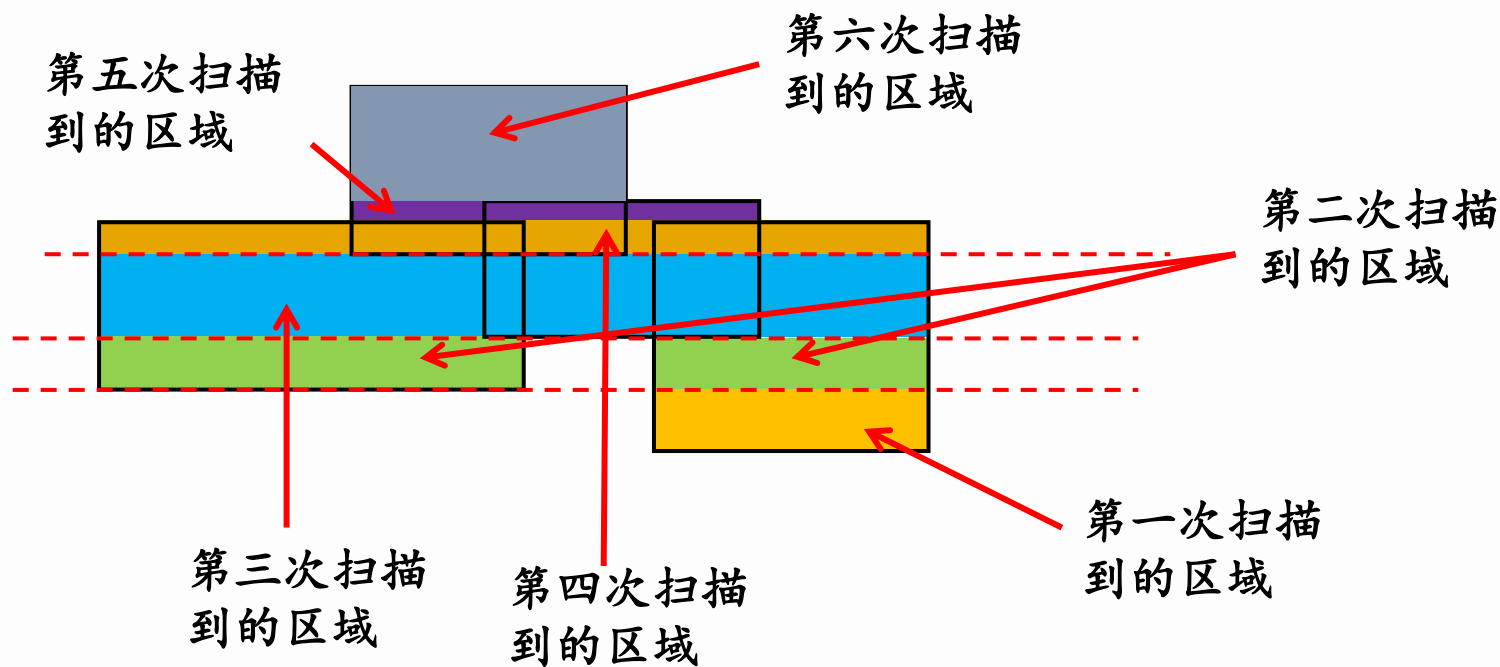
1、离散化

由于坐标特别大，空间会炸掉，所以我们需要离散化，如果我们的扫描线是横向的，那么我们把横坐标离散化，如果是纵向的，把纵坐标离散化

2、保存边

存储这条边，由于是平行线，所以只需要存储左右点的x坐标，和一个y坐标就可以了，再用1和-1记录是起点边还是终点边。

矩形面积之和



3、把边按照y值从小到大排序，然后从下到上扫描上下边，扫描到下边时，区间值+1，扫描到上边时，区间值-1，，用（下一条边-上一条边）*此时区间有效长度（非0的个数）来计算面积。

实现过程

第一步：先重新开两个数组来记录，一个数组记录横着的每一条线 $(y, x1, x2)$ ，以及这条线是矩形区域的进入边还是出去边，进入边，表示长度增加，用1表示，出去边表示已有的长度会较少，用-1表示，另一个数组记录则每一个 x 的坐标。然后再从小到大排序

```
for(int i=1;i<=n;i++)
{
    scanf("%lf%lf%lf%lf",&x1,&y1,&x2,&y2);
    k++;
    p[k].y=y1;p[k].x1=x1;p[k].x2=x2;p[k].flag=1;//进入边
    x[k]=x1;
    k++;
    p[k].y=y2;p[k].x1=x1;p[k].x2=x2;p[k].flag=-1;
    x[k]=x2;
}
```

实现过程

第二步：建线段树，但是要注意，该题实际上是边权，而不是点权，比如 $[1, 2]$ 实际上表示长度为1的线段，不能再分了， $[1, 1]$ 根本不是表线段长度，并且，如果用 $[1, 2]$ 和 $[3, 4]$ 表示，那么很明显 $[2, 3]$ 长度丢失了. 并且线段树还需要记录区间实际左右端点位置。

```
tree[id].l=l;tree[id].r=r;
tree[id].fl=x[l];tree[id].fr=x[r];
tree[id].cf=tree[id].flen=0;//一定要有，因为多组数据
if(l+1==r)//因为是边权，所以线段树和普通线段树有点不一样
return ;
int mid=(l+r)/2;
build(id*2,l,mid);
build(id*2+1,mid,r);//如果是mid和mid+1,那么就少了一截
```

实现过程

第三步：修改。每次加入一条边，总的长度就会发生改变，但是要注意重叠的部分只算一次，所以我们还需要记录每个位置到底重复了多少次，比如 $[1, 10]$ 重复了三次，那么下次出现 $[1, 10]$ 的出边的时候， $[1, 10]$ 实际上还是被覆盖的，并不是直接就没有了。

同时我们还需要更新当前这段区间的覆盖长度，如果区间整体被覆盖，则就为区间实际长度，如果没有整体被覆盖，那么如果是叶子结点，就是0(没有被覆盖)，如果不是叶子结点，那么就等于他的左右儿子的覆盖长度之和。更新的时候一定要注意

实现过程

```
void update(int id, point a)
{
    if(tree[id].fl==a.x1&&tree[id].fr==a.x2)
    {
        tree[id].cf+=a.flag;
        js(id); // 计算覆盖长度, 重复的只算一次
        return ;
    }
    if(a.x2<=tree[id*2].fr) // 如果该线段都在左区间
        update(id*2, a);
    else if(a.x1>=tree[id*2+1].fl) // 如果都在右区间
        update(id*2+1, a);
    else // 如果两个区间各一部分
    {
        point t1=a, t2=a;
        t1.x2=tree[id*2].fr;
        update(id*2, t1);
        t2.x1=tree[id*2+1].fl;
        update(id*2+1, t2);
    }
    js(id); // 一定不要忘记更新根结点的信息
}
```



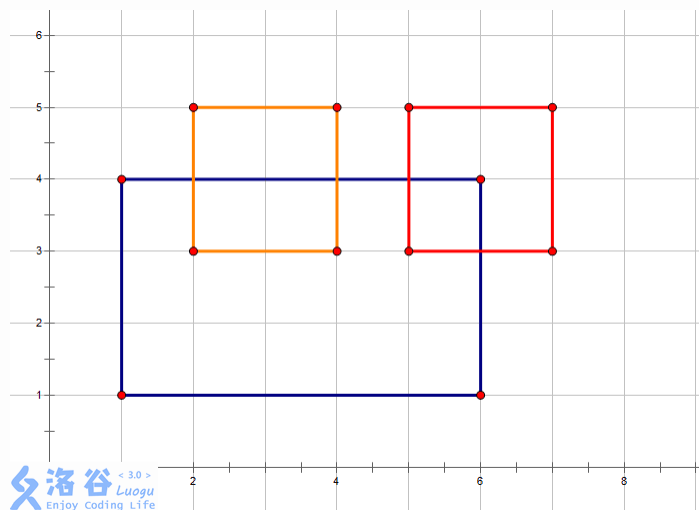
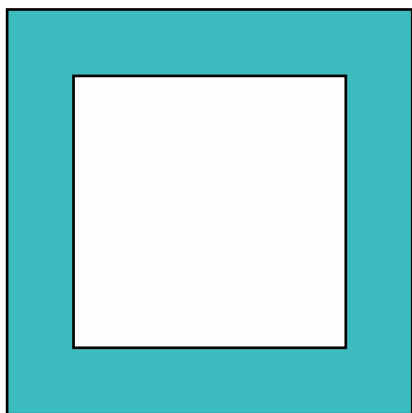
实现过程

第四步：查询答案，我们发现根本不需要写线段树的查询函数，因为每次都是求总的长度，所以直接找到根结点的覆盖长度就是答案，然后再乘以高度只差就是当前两条线段之间的面积。



矩形周长之和

上题问面积之和是多少，那么如果问你周长之和呢？周长也就是围成的形状的表面的边长之和（图中黑色线条为周长）。



矩形周长之和

首先，矩形的周长分为长和宽，如果我们按照前面求面积的方法来做，宽度就不太好求，所以我们可以把矩形的长和宽分开来求解，即先求水平的，再求垂直的。

在考虑的时候，我们发现要分上底面和下底面考虑。

对于下底面的边 $[s, t]$ ，先查询 $[s, t]$ 之间被覆盖的长度 x ，然后 $t-s-x$ 就是新增的

对于上底面的边 $[s, t]$ ，先删除 $[s, t]$ 对应的边，然后再查询 $[s, t]$ 之间被覆盖的长度 x ，然后 $t-s-x$ 就是。



练习题

洛谷 1502

洛谷 4125

洛谷 1151

洛谷 1856

P0J 1151

P0J 2932

P0J 1177

